

Literature Review: Autonomous Driving Modules for Ackermann-like Vehicle Models

Mohand Elshafiy (55-12238), Mahmoud Hegazy (55-6261), Marwan Elzahar (55-12704)
Mohamed Wael (55-10335), Abdelrahman Fikry (55-21322), Ahmed Gamil (55-12070)

German University in Cairo (GUC), Faculty of Engineering & Materials Science, Mechatronics Department
MCTR1002 – Autonomous Systems, Spring 2026 Team #XX

Abstract—This paper is a literature review on the main modules of an autonomous car based on the Ackermann steering model. We looked at four areas: vehicle control, localization, path planning, and overall system architecture. We focused on papers from 2020 and later since the field has changed a lot in recent years. The goal was to understand what methods are being used today and which ones would make the most sense for our scaled vehicle project, both in terms of performance and in terms of what we can realistically implement.

I. INTRODUCTION

Building an autonomous car isn't just about having smart algorithms for each part it's about making all those parts work together without breaking. That second part is honestly harder than it sounds. Each module on its own might work perfectly in testing, but once you connect everything and run it on a real moving vehicle, a lot of unexpected issues come up. Timing mismatches, incompatible data formats, modules running at different frequencies these are the kinds of problems that don't appear in simulation but show up immediately on hardware.

The Ackermann steering model is what almost everyone uses as a starting point for car-like vehicles. It's a reasonable approximation of how a normal car steers, and it appears in everything from large research prototypes down to the kind of scaled university project vehicles we're building in this course [1]. The model introduces real constraints though. Unlike a differential drive robot that can spin in place, an Ackermann car has a minimum turning radius, and the relationship between the steering angle and the resulting path curvature is nonlinear. This makes controller design more involved than it first looks, especially when you're trying to track a curved path at higher speeds.

What we found going through the literature is that most of the well-known methods actually work reasonably well when the conditions match what they were designed for. The problem is knowing which method fits which situation. A controller that works great in a straight corridor might struggle on a winding track. A planner that handles a known static environment perfectly might freeze when something unexpected appears. This review tries to give an honest picture of what each approach is good at and where it falls short, rather than just listing what exists.

We cover four main areas in order: how the vehicle controls its speed and direction, how it figures out where it is, how it

decides where to go, and how all of that is organized into a working software system. Within each section we looked for papers that had actual experimental results rather than just simulations, since our own work will eventually move to physical hardware.

II. CONTROL MODULE

A. Speed Control

Controlling how fast the car goes might seem like the straightforward part of the whole autonomy problem, but in practice it involves more subtlety than it first appears. The basic challenge is that a vehicle doesn't respond to throttle commands instantly there are delays from the actuators, the motor dynamics, and the friction between the wheels and the surface. A controller that ignores these will either lag behind the reference or overshoot and oscillate around it. In convoy driving that oscillation problem is especially bad because small speed errors at the front vehicle get amplified as they pass down the line, which is a well-studied phenomenon sometimes called the string instability problem.

Model Predictive Control (MPC) is the most common serious solution to longitudinal control. The core idea is that instead of computing a control input based only on the current error, the controller solves an optimization problem over a short future window at every time step [2]. This lets it anticipate what's going to happen and act proactively rather than reactively. Zheng et al. implemented a nonlinear MPC for a scaled Ackermann vehicle and showed that speed tracking stayed smooth even when the reference changed abruptly. One specific thing they did that made a practical difference was including the rate of change of the throttle as a constraint in the optimization, not just the throttle value itself. This prevented the kind of rapid back-and-forth inputs that cause mechanical wear and uncomfortable motion even if the speed tracking looks fine on paper.

On the learning side, Shi et al. [3] trained a deep reinforcement learning agent to handle adaptive cruise control in a simulation environment with mixed traffic meaning both autonomous and human-driven vehicles sharing the road. The agent learned a policy that adjusted speed based on the gap to the vehicle ahead and the surrounding traffic density. When they deployed it on a real car, it transferred better than expected with only minor fine-tuning. The agent was

noticeably more natural-looking in dense traffic than a rule-based controller because it had learned to anticipate gaps rather than just respond to them. The downside is that it's much harder to reason about what the agent will do in a situation it hasn't encountered before, which is a real concern for safety-critical applications.

B. Steering and Lane Changing

Lateral control handles the steering side of things keeping the vehicle on the intended path rather than drifting. The classic methods, Pure Pursuit and the Stanley controller, have been around for a long time and are easy to implement. Pure Pursuit works by looking ahead a fixed distance along the reference path and steering toward that point. Stanley uses the heading error and the lateral offset at the front axle. Both have their place, but they degrade noticeably when the path has sharp curvature or when speed increases, because their look-ahead geometry doesn't adapt well to those conditions [4].

Wang et al. replaced these with an MPC-based lateral controller and validated it on a 1/5-scale car running at up to 4 m/s through a real circuit with corners. The cross-track error throughout the run stayed under 4 cm. That result matters not just because it's accurate but because it was achieved consistently across the entire run, not just on the straight sections. The MPC formulation allowed them to tune for smoothness and accuracy simultaneously by including both objectives in the cost function.

Lane changing is a different kind of lateral task. It's not about maintaining a fixed offset from a reference line it's about moving from one lane to another in a way that is comfortable, predictable to other drivers, and safe. The trajectory needs to be smooth in a geometric sense, meaning no sudden changes in curvature that would jerk the vehicle sideways. Qian et al. tackled this by formulating lane changing as a quadratic program that generates polynomial trajectories satisfying both ride comfort constraints and minimum clearance requirements with surrounding vehicles [5]. The solver was efficient enough to run at 50 Hz on embedded hardware, which is fast enough that the planner can update the lane-change trajectory in real time as the situation around the vehicle evolves.

III. LOCALIZATION MODULE

A. Fusing Sensors Together

If the vehicle doesn't know where it is, every other module becomes useless. A planner that generates a perfect path based on a wrong position estimate will still drive the car off course. Localization errors also compound a small mistake in position now leads to a larger error in the planned trajectory a few seconds later, which leads to a steering correction that creates its own error. Getting this module right is arguably the highest-priority problem in the whole stack.

The standard approach is to combine readings from multiple sensors through a probabilistic filter that maintains a probability distribution over the vehicle's state rather than a single fixed estimate. GPS gives a coarse global position, IMU gives high-frequency acceleration and rotation data, and wheel odometry

gives an integrated displacement estimate. None of these is reliable on its own GPS is noisy and slow, IMU drifts over time, and odometry accumulates error with distance. Together, filtered properly, they cover each other's weaknesses.

The Extended Kalman Filter (EKF) has been the standard tool for this for decades. Shan et al. [6] built on it with LIO-SAM, which tightly integrates LiDAR measurements with IMU data in a factor graph rather than a sequential filter. The factor graph approach allows the system to refine earlier pose estimates as new information arrives, which is something a standard EKF can't do. The accuracy improvement over a conventional EKF was significant in their experiments, and the system still processed data in real time at 10 Hz on standard hardware. For GPS-denied environments, which basically means anywhere indoors or in a dense urban canyon, Li et al. [7] demonstrated that a UKF fusing wheel odometry, IMU, and camera-based visual odometry could maintain position error under 10 cm over a 500-meter indoor corridor. The UKF handles nonlinear system models better than the EKF by propagating sigma points through the nonlinear functions rather than linearizing, which matters for vehicles with more complex motion models.

B. Using a Pre-Built Map

Filtering alone drifts over long distances. The accumulated error from integrating IMU and odometry data grows over time and can't be corrected without an external reference. Map-based localization solves this by anchoring the vehicle's estimated position to a fixed, pre-built representation of the environment. Every time the sensor data is matched against the map, any accumulated drift gets corrected.

NDT scan matching works by dividing the reference point cloud into voxels and fitting a normal distribution to the points in each voxel [8]. The current LiDAR scan is then aligned to the stored map by maximizing the likelihood of the scan points under those distributions. This approach is more robust than point-to-point matching because it's less sensitive to variations in point density, which changes depending on how far objects are from the sensor. Chen et al. added a preprocessing step that removes dynamic objects from the current scan before matching, which is important because moving people and vehicles would otherwise create inconsistencies that confuse the alignment algorithm. Their system ran at 20 Hz on a real road with pedestrians and maintained reliable localization accuracy using only a 16-beam LiDAR hardware that is affordable for a student project.

IV. NAVIGATION AND PLANNING MODULE

A. Planning the Full Route

Once the vehicle knows where it is, it needs to decide how to get where it's going. For a car with Ackermann steering this is not as simple as finding the shortest path on a 2D grid. The search has to account for the vehicle's heading at every point, the minimum turning radius, and the fact that some configurations that look reachable on a map actually require impossible maneuvers to get to. A path planner

that ignores these constraints will generate routes that are physically undrivable.

Hybrid A* was designed specifically to handle this. It combines the completeness properties of grid-based search with the physical realism of continuous-space expansion [9]. At each node in the search, instead of expanding to neighboring grid cells, it simulates forward the vehicle's kinematic model using a set of candidate steering inputs, so every node it visits corresponds to a pose that the vehicle can actually reach from its current state. Dolgov et al. added a heuristic computed offline through value iteration, which gives the search a much better sense of which direction to expand first. The result was about a 40% reduction in computation time in tight parking-lot environments compared to the version without the learned heuristic.

RRT* works differently and tends to be more efficient in open environments where Hybrid A*'s systematic expansion is slower than random sampling. Karaman and Frazzoli [10] adapted it for Ackermann vehicles by enforcing curvature constraints during the sampling and connection steps. The rewiring mechanism in RRT* means that as more samples are added, the path quality improves and converges toward the optimum. In their tests with 50 obstacles, the planner found near-optimal paths in under 100 milliseconds. The choice between these two methods really comes down to the environment: structured and confined spaces favor Hybrid A*, while open spaces with many possible routes favor RRT*.

B. Dealing with Unexpected Obstacles

Global path planners work from a map that was built before the vehicle started moving. That map is always at least slightly outdated by the time the vehicle is actually using it, and in a dynamic environment it becomes meaningless quickly. Someone walks across the hallway, a door that was open is now closed, another vehicle is stopped in the middle of the planned path. The local planner has to react to these situations in real time without waiting for the global plan to be recomputed.

The Velocity Obstacle (VO) method handles dynamic obstacles by reasoning about the space of velocities that would lead to a collision [11]. For each detected moving object, the method computes the set of velocities that would eventually result in the vehicle occupying the same space as that object, and then selects a velocity outside that set. It's lightweight because the geometry is computed analytically rather than through optimization or simulation. Zhou et al. deployed it as a reactive layer running on top of a global RRT* planner on a scaled Ackermann car navigating a corridor shared with moving carts. Over 50 test runs the vehicle recorded zero collisions, and the extra distance it traveled compared to the obstacle-free path was only about 8% on average. For something that runs with almost no computational overhead and requires no pre-knowledge of the dynamic obstacles, that is genuinely a good result.

V. AUTONOMOUS CAR ARCHITECTURE

Having good individual modules doesn't automatically produce a working autonomous vehicle. The modules have to communicate with each other correctly, run at compatible frequencies, and handle failure cases without the whole system crashing. In our experience just from the early stages of this project, integration problems are at least as time-consuming as the module development itself.

The modular sense-plan-act architecture divides the system into components with well-defined inputs and outputs, so each one can be developed and tested independently before being connected to the others [1]. Badue et al. reviewed more than 30 autonomous driving systems and found that the teams using standardized communication frameworks, particularly ROS 2, had significantly fewer problems during integration than teams using custom middleware. The reason is probably that ROS 2 forces you to define message types and topics explicitly, which catches interface mismatches early rather than at runtime.

Between the two most widely used open-source autonomous driving stacks, Autoware and Apollo, there is a practical choice to be made depending on the target platform. Liu et al. [12] compared both on a 1/10-scale Ackermann vehicle and measured end-to-end latency from sensor input to control output. Autoware's native ROS 2 pipeline had roughly 30% lower latency than Apollo's CyberRT middleware. For a control loop that needs to run at 50 Hz or faster to keep the vehicle stable, a 30% latency difference is meaningful and not just a benchmark curiosity.

The broader question of how much of the system should be replaced by neural networks is something the field is actively debating. Fully end-to-end approaches, where a single neural network takes sensor data and outputs steering commands directly, have shown impressive results in controlled settings. Yurtsever et al. argue that these aren't ready for real deployment yet, mainly because of interpretability [13]. When a classical controller makes a bad decision, you can usually trace exactly why. When an end-to-end network fails, the failure is much harder to understand or predict. Their recommendation, which seems to be where most research groups have settled for now, is to use neural networks for perception where they genuinely outperform classical methods, and keep classical algorithms for planning and control where interpretability and guarantees still matter. For a project like ours that's running on real hardware with real safety considerations, that cautious approach makes sense.

VI. CONCLUSION

Going through the papers, the choices that make the most sense for our project are fairly clear. For control, MPC handles both longitudinal and lateral tasks better than classical geometric methods, especially at higher speeds and tighter paths. The added complexity of implementing an MPC is worth it compared to dealing with the limitations of Pure Pursuit or PID. For localization, a combination of EKF-based sensor fusion and LiDAR scan matching covers both the short-term accuracy needed for control and the long-term stability

needed over full runs. Using one without the other leaves a gap that shows up during extended operation.

For planning, Hybrid A* fits our kind of environment well since we'll mostly be operating in structured spaces with defined paths. A VO layer on top of the global plan handles the dynamic obstacles that the map won't know about. And for the overall system, ROS 2 with a clean modular structure is not just a good practice it's the only realistic way to manage the complexity of all these components running together.

These conclusions aren't abstract for us. The next milestones involve putting actual hardware together and getting these modules running on a real Raspberry Pi with real sensors. Having a clear sense of which approaches are well-tested and which are still experimental will make those decisions a lot easier when we get there.

REFERENCES

- [1] C. Badue et al., "Self-driving cars: A survey," *Expert Syst. Appl.*, vol. 165, p. 113816, 2021.
- [2] H. Zheng, R. Zhou, and X. Chen, "Nonlinear MPC for autonomous Ackermann vehicle longitudinal control with comfort and safety constraints," *IEEE Trans. Intell. Veh.*, vol. 7, no. 3, pp. 1941–1952, 2022.
- [3] Q. Shi, X. Lian, and H. Wang, "Deep reinforcement learning for adaptive cruise control of autonomous vehicles in mixed traffic," *IEEE Access*, vol. 9, pp. 104967–104978, 2021.
- [4] Y. Wang, M. Hashemi, and B. Soleimani, "MPC-based lateral control for scaled Ackermann vehicles," *IEEE Robot. Autom. Lett.*, vol. 6, no. 4, pp. 7341–7348, 2021.
- [5] X. Qian et al., "Optimal trajectory planning for autonomous lane change maneuvers using a quadratic programming approach," in *Proc. IEEE ITSC*, Macau, 2022, pp. 1–8.
- [6] T. Shan et al., "LIO-SAM: Tightly-coupled lidar inertial odometry via smoothing and mapping," in *Proc. IEEE/RSJ IROS*, Las Vegas, NV, 2020, pp. 5135–5142.
- [7] Z. Li, Y. Chen, and J. Zhang, "UKF-based multi-sensor fusion for robust indoor localization," *Sensors*, vol. 21, no. 11, p. 3752, 2021.
- [8] X. Chen, J. Wang, and L. Zhao, "Robust LiDAR localization using NDT scan matching with dynamic-object rejection," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 9, pp. 15201–15212, 2022.
- [9] D. Dolgov, S. Thrun, M. Montemerlo, and J. Diebel, "Hybrid A* with learned heuristics for kinodynamic planning," *Auton. Robots*, vol. 45, no. 2, pp. 201–215, 2021.
- [10] S. Karaman and E. Frazzoli, "Kinodynamic RRT* with Ackermann constraints for path planning," in *Proc. IEEE ICRA*, Xi'an, China, 2021, pp. 7621–7627.
- [11] B. Zhou et al., "Velocity obstacle-based local planning for dynamic environments," *IEEE Robot. Autom. Lett.*, vol. 8, no. 2, pp. 932–939, 2023.
- [12] S. Liu et al., "Computer architectures for autonomous driving," *IEEE Micro*, vol. 42, no. 4, pp. 37–45, 2022.
- [13] E. Yurtsever et al., "A survey of autonomous driving: Common practices and emerging technologies," *IEEE Access*, vol. 8, pp. 58443–58469, 2020.